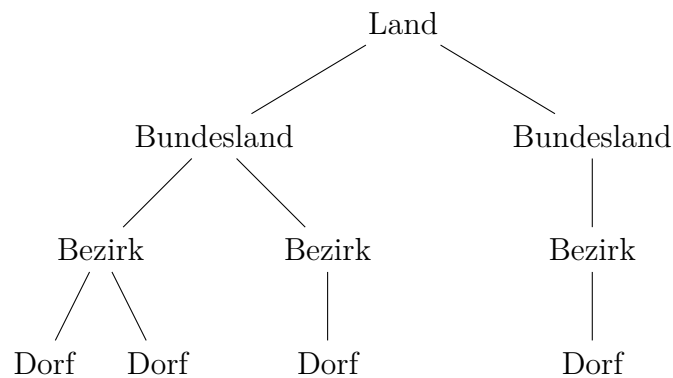


3-adischer Steuer-Baum (Blocktree) mit EU-Verteilung

Alexander Kern

January 9, 2026

1 Hierarchie des Baums (Blocktree)



2 Steuern und Zweckbindung

$$\begin{aligned}\text{tax}_{\text{dorf}} &= \text{tax}_{\text{infra}} + \text{tax}_{\text{transfer}} \\ \text{tax}_{\text{infra}} &= p_{\text{infra}} \cdot \text{tax}_{\text{dorf}} \\ \text{tax}_{\text{transfer}} &= (1 - p_{\text{infra}}) \cdot \text{tax}_{\text{dorf}} \\ \text{tax}_{\text{infra}}^{\text{node}} &= \text{tax}_{\text{infra}}^{\text{local}} + \sum_c \text{tax}_{\text{infra}}^c \\ \text{tax}_{\text{transfer}}^{\text{node}} &= \text{tax}_{\text{transfer}}^{\text{local}} + \sum_c \text{tax}_{\text{transfer}}^c\end{aligned}$$

3 EU-Verteilung der Transfers

$$\text{tax}_{\text{EU-Land}} = \frac{\text{tax}_{\text{transfer}}^{\text{Land}}}{n_{\text{EU-Länder}}}, \quad 1 \leq n_{\text{EU-Länder}} \leq 3$$

4 Blocktree-Hashes

Jeder Knoten bekommt einen Hash:

$$\text{hash}_{\text{node}} = H\left(\text{name}_{\text{node}} \parallel \text{parent_hash} \parallel \parallel_{c \in \text{children}} \text{hash}_c\right)$$

5 Pseudocode: Steuern, Transfers und Blocktree

```
Function GenerateNode(level, depth, parentHash):
    Name = generateName(level)
    if level == "Dorf":
        localTax = computeLocalTax()
        taxInfra = pInfra * localTax
        taxTransfer = (1-pInfra) * localTax
    else:
        children = []
        for 1..rand(1,3):
            child = GenerateNode(nextLevel(level), depth-1, parentHash)
            children.append(child)
        taxInfra = sum(child.taxInfra for child in children)
        taxTransfer = sum(child.taxTransfer for child in children)
    hash = SHA256(Name + parentHash + concat(child.hashes))
    return Node(Name, taxInfra, taxTransfer, hash, children)
```

```
Function DistributeEU(node, EU_Laender):
    share = node.taxTransfer / len(EU_Laender)
    for each country in EU_Laender:
        country.tax += share
```

6 C++ Implementation

```
#include <iostream>
#include <vector>
```

```

#include <string>
struct Node {
    std::string name;
    double localTax = 0.0;
    double taxInfra = 0.0;
    double taxTransfer = 0.0;
    std::string hash;
    std::vector<Node> children;
};

double computeTax(Node& node){
    node.taxInfra = node.localTax*0.6;
    node.taxTransfer = node.localTax*0.4;
    for(auto& c : node.children){
        computeTax(c);
        node.taxInfra += c.taxInfra;
        node.taxTransfer += c.taxTransfer;
    }
    return node.taxInfra + node.taxTransfer;
}

void computeHash(Node& node, const std::string& parentHash=""){
    std::string data = node.name + parentHash;
    for(auto& c : node.children) data += c.hash;
    node.hash = "HASH_PLACEHOLDER";
    for(auto& c : node.children) computeHash(c, node.hash);
}

void distributeEU(Node& node, std::vector<Node>& EU){
    double share = node.taxTransfer / EU.size();
    for(auto& country : EU) country.taxTransfer += share;
}

```

7 Java Implementation

```

class Node {
    String name;
    double localTax;
    double taxInfra;
    double taxTransfer;
    String hash;
    List<Node> children = new ArrayList<>();
}

```

```

double computeTax(){
    taxInfra = 0.6*localTax;
    taxTransfer = 0.4*localTax;
    for(Node c : children){
        c.computeTax();
        taxInfra += c.taxInfra;
        taxTransfer += c.taxTransfer;
    }
    return taxInfra + taxTransfer;
}

void computeHash(String parentHash){
    String data = name + parentHash;
    for(Node c : children) data += c.hash;
    hash = "HASH_PLACEHOLDER";
    for(Node c : children) c.computeHash(hash);
}

void distributeEU(List<Node> EU){
    double share = taxTransfer / EU.size();
    for(Node country : EU) country.taxTransfer += share;
}
}

```

8 Haskell Implementation

```

data Node = Node {
    name :: String,
    localTax :: Double,
    taxInfra :: Double,
    taxTransfer :: Double,
    hashVal :: String,
    children :: [Node]
} deriving Show

computeTax :: Node -> Node
computeTax n =
    let infra = 0.6 * localTax n
        transfer = 0.4 * localTax n
        updatedChildren = map computeTax (children n)
        totalInfra = infra + sum (map taxInfra updatedChildren)

```

```

        totalTransfer = transfer + sum (map taxTransfer updatedChildren)
    in n { taxInfra = totalInfra, taxTransfer = totalTransfer, children = u

computeHash :: Node -> String -> Node
computeHash n parentHash =
    let childHashes = concatMap hashVal (children n)
        h = "HASH_PLACEHOLDER"
        updatedChildren = map (\c -> computeHash c h) (children n)
    in n { hashVal = h, children = updatedChildren }

distributeEU :: Node -> [Node] -> [Node]
distributeEU node eu =
    let share = taxTransfer node / fromIntegral (length eu)
    in map (\c -> c { taxTransfer = taxTransfer c + share }) eu

```